



Quick Start

This primer shows you how to put a map on a web page. The development setup uses [Node](#) (14 or higher) and requires that you have [git](#) installed.

Set up a new project

The easiest way to start building a project with OpenLayers is to run `npm create ol-app` :

```
npm create ol-app my-app  
cd my-app  
npm start
```

The first command will create a directory called `my-app` (you can use a different name if you wish), install OpenLayers and a development server, and set up a basic app with `index.html` , `main.js` , and `style.css` files.

The second command (`cd my-app`) changes the working directory to your new `my-app` project so you can start working with it.

The third command (`npm start`) starts a development server so you can view your application in a browser while working on it. After running `npm start` , you'll see output that tells you the URL to open. Open <http://localhost:5173/> (or whatever URL is displayed) to see your new application.

Exploring the parts

An OpenLayers application is composed of three basic parts:

- The HTML markup with an element to contain the map (`index.html`)
- The JavaScript that initializes the map (`main.js`)
- The CSS styles that determine the map size and any other customizations (`style.css`)

The markup



```
<!/DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, ini
    <title>Quick Start</title>
  </head>
  <body>
    <div id="map"></div>
    <script type="module" src="./main.js"></script>
  </body>
</html>
```

The two important parts in the markup are the `<div>` element to contain the map and the `<script>` tag to pull in the JavaScript. The map container or target should be a block level element (like a `<div>`) and it must appear in the document before the `<script>` tag that initializes the map.

The script

Open the `main.js` file in a text editor. It should look something like this:

```
import './style.css';
import Map from 'ol/Map.js';
import OSM from 'ol/source/OSM.js';
import TileLayer from 'ol/layer/Tile.js';
import View from 'ol/View.js';

const map = new Map({
  target: 'map',
  layers: [
    new TileLayer({
      source: new OSM(),
    }),
  ],
  view: new View({
    center: [0, 0],
    zoom: 2,
  }),
});
```

OpenLayers is packaged as a collection of [ES modules](#). The `import` lines are used to pull in the modules that your application needs. Take a look through the [examples](#) and [API docs](#) to understand which modules you might want to use.



OpenLayers

imported from JavaScript modules. If you were using a different development server, you might include the `style.css` in a `<link>` tag in the `index.html` instead.

The `main.js` module serves as an entry point for your application. It initializes a new map, giving it a single layer with an OSM source and a view describing the center and zoom level. Read through the [Basic Concepts tutorial](#) to learn more about `Map`, `View`, `Layer`, and `Source` components.

The style

Open the `style.css` file in a text editor. It should look something like this:

```
@import "node_modules/ol/ol.css";

html,
body {
  margin: 0;
  height: 100%;
}

#map {
  position: absolute;
  top: 0;
  bottom: 0;
  width: 100%;
}
```

The first line imports the `ol.css` file that comes with the `ol` package (OpenLayers is published as the [ol package](#) in the npm registry). The `ol` package was installed in the `npm create ol-app` step above. If you were starting with an existing application instead of using `npm create ol-app`, you would install the package with `npm install ol`. The `ol.css` stylesheet includes styles for the elements that OpenLayers creates – things like buttons for zooming in and out.

The remaining rules in the `style.css` file make it so the `<div id="map">` element that contains the map fills the entire page.

Deploying your app



OpenLayers

development server (with `npm start`). After you have finished making edits, it is time to bundle or build your application so that it can be deployed as a static website (without needing to run a development server like Vite).

To build your application, run the following:

```
npm run build
```

This will create a `dist` directory with a new `index.html` and assets that make up your application. These `dist` files can be deployed with your production website.

Code licensed under the [2-Clause BSD](#). All documentation [CC BY 3.0](#). Thanks to our [sponsors](#).
[This site is powered by Netlify](#).